

ESPRESSIONI

↳ categoria sintattica usata per manipolare/
denotare valori

⇒ Vengono valutate per
restituire un valore (dato)

dati su
cui opera
i.e. PL

uguaglianza
sintattica

(poco interessante)

→ espressioni
uguali sono
sempre equivalenti

equivalenza
semantica

Due espressioni sintatticamente
diverse sono equivalenti
se a partire da qualunque
stato della macchina su cui
sono valutate denotano lo
stesso valore

$$e_1 = 2 + 4 \quad \rightsquigarrow 6$$

derivate

$$e_2 = 3 + 2$$

$$e_3 = 5 + 1$$

⋮

$e_1 \neq e_2 \neq e_3$
(sintatticamente
diverse)

$e_1 \equiv e_2 \equiv e_3$
semanticamente
equivalenti

Valori esprimibili $\rightsquigarrow$ insieme dei
valori che
possiamo denotare
con le espressioni

Eval

Nel linguaggio IMP $\Rightarrow$ Eval = bool $\cup$ int

↳ bool = {tt, ff}

↳ int = {m | m $\in \mathbb{Z}$ }

Aspetti che caratterizzano le espressioni

→ operatori utilizzati, argomenti, notazione
(prefissa, infissa o postfissa)

$$\begin{array}{c} 5 + 3 \\ \hline \text{square}(\underline{5}) \end{array}$$

→ Regole di precedenza $5 + 3 * 4$ $*$ e $/$ prec. su $+$ e $-$

→ Regole di ~~associatività~~ (da sx verso dx)

$$\underline{5 + 3} - 2$$

→ Ordine di valutazione degli operandi

$$\begin{array}{c} x \wedge y \\ \downarrow \\ \text{ff} \Rightarrow \text{ff} \end{array}$$

→ Presenza di side-effects $x++$

→ Overloading degli operatori

→ Espressioni con tipi misti $5 + \text{ciao}$



NOTAZIONE

↓

$$\begin{array}{c} a + b \\ \text{infissa} \end{array}$$

↓

$$\begin{array}{c} + a b \\ \text{prefissa} \end{array}$$

↓

$$\begin{array}{c} a b + \\ \text{postfissa} \end{array}$$

⇒ la notazione ha effetto sulle regole di precedenza e associatività

Algoritmo di valutazione di notazione postfixa

↳ use una pila

5 3 +

1. → Legge il prossimo simbolo

se simbolo = operatore

↳ applica l'operatore
sui operandi sulla pila (in numero
pari alla arità dell'operatore)

↳ elimine gli operandi usati
dalla pila

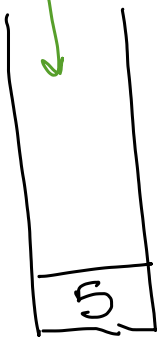
↳ salva risultato sulla pila

se simbolo = operando

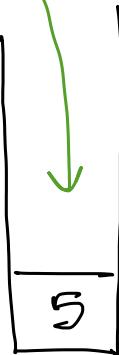
↳ copia il simbolo sulla pila

Se l'espressione non è stata completata
ripeti da 1.

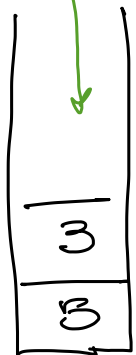
~~5~~ 3 2 * +



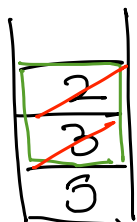
~~3~~ 2 * +



~~2~~ * +



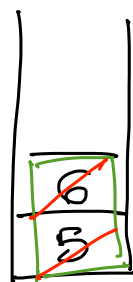
~~*~~ + arità = 2



↳ $2 * 3 = 6$

6

~~+~~ arità = 2



⇒ $6 + 5 = 11$

11

Algoritmo valutazione espressioni in notazione prefissa

Nasce con l'espressione da valutare + più

1. Legge il prossimo simbolo e push nella pila

se simbolo = operatore $C_{op} = n$ (argomenti)

torna 1.

se simbolo = operando $C_{op}--$

4. se $C_{op} \neq 0$ torna 1.

se $C_{op} == 0$

applica l'operatore "più in alto" nella pila
agli operandi che gli stanno sopra

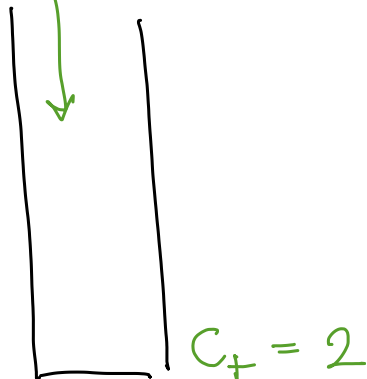
sistema
contato per
l'operatore
precedente
nella pila

se non ci sono simboli nella pila va a 6.
altrimenti $C_{op} = n - m$ $\rightarrow$ n ovvero operatore op'
(operatore precedente) $\rightarrow$ m n° operandi sulla pila
sopra op'

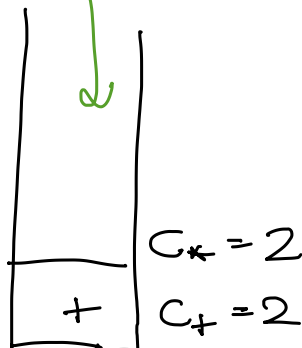
torna a 4

6. Se la pila $\neq$ vuoto torna a 1.

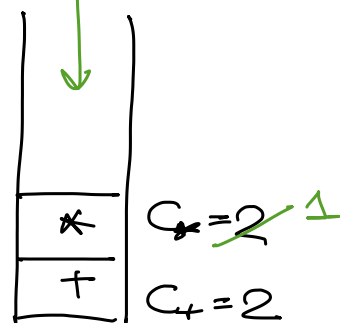
~~+~~ * 3 2 5

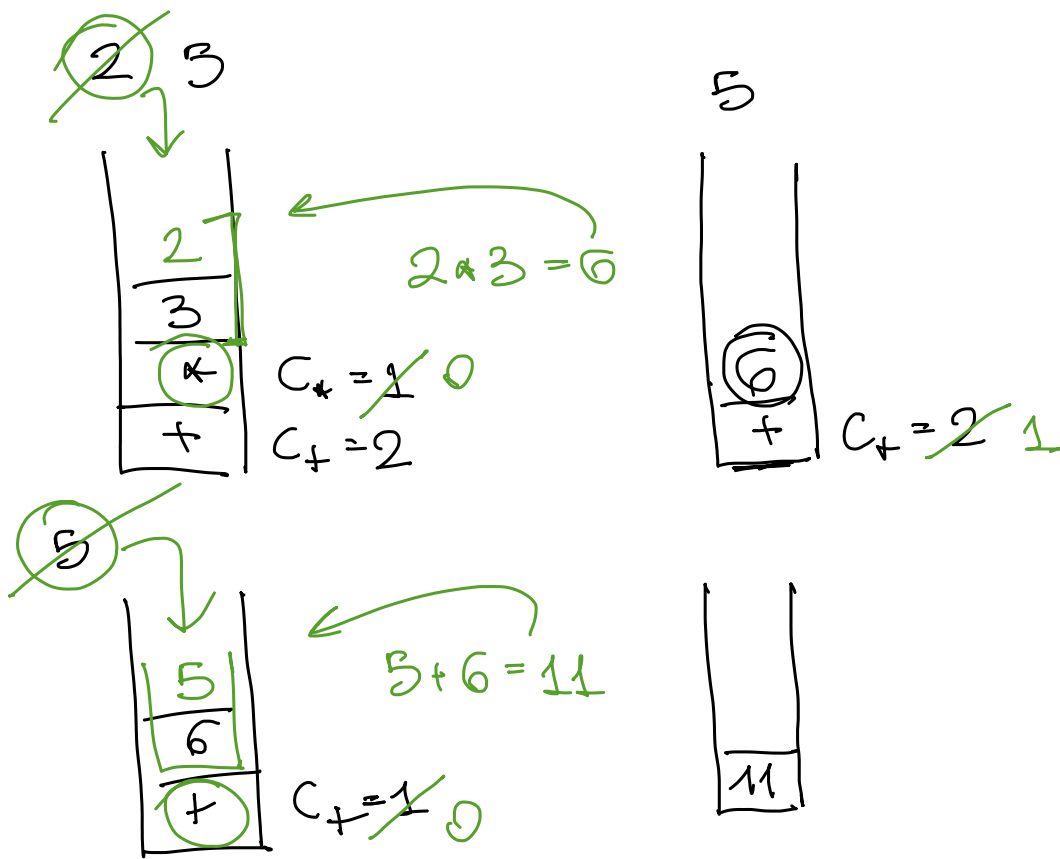


~~*~~ 3 2 5



~~3~~ 2 5





Notazione infissa $\rightarrow$ Richiede regole di precedenza e associatività

Precedenza
 $\downarrow$

() (parentesi)

operatori unari

not e and b

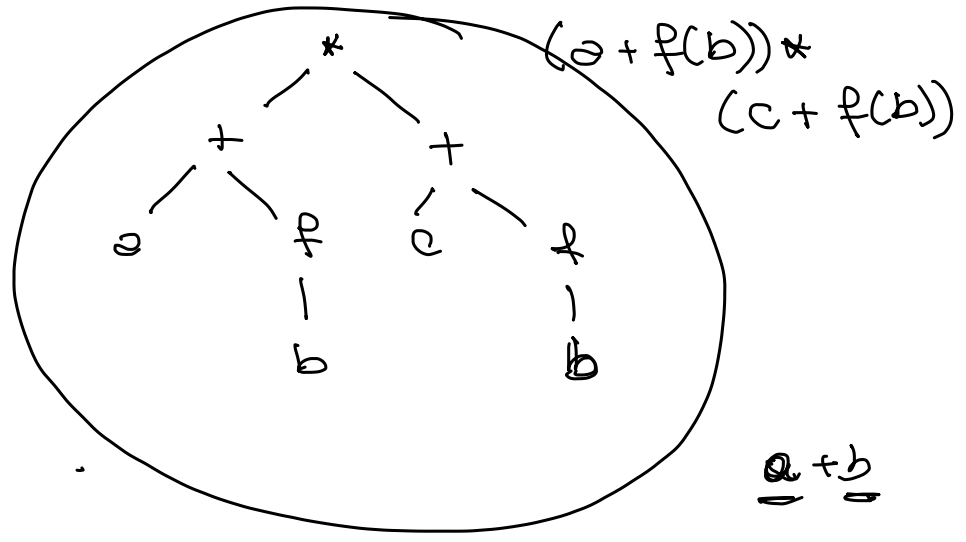
** (esponentiale)

*, /

+ -

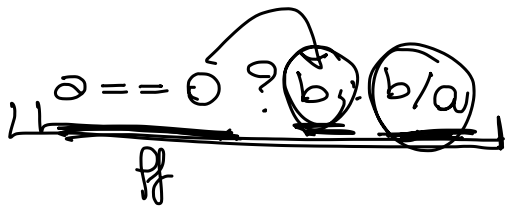
OPERANDI NON DEFINITI → Valutazione espressioni

Espressioni → rappresentate come alberi



Problemi:

- Operandi non definiti
- Antmerco finita
- Effetti collaterali



→ Valutazione easy OK
altrimenti NO

a and b

a or b

if $a == 0$
then $x = b$
else $x = b/a$

```
[ p := lista;
  while (p != nil) and (p.valore != 3) do
    p := p.prossimo;
```

Effetti collaterali:

$a = 10;$
 $b = a + \text{fun}(a);$

~~OK~~
 $a \rightarrow 10$
prima voluto a e poi fun
 $b = 10 + \text{fun}(a)$

fun(a) → modifica a = 100

$b = 100 + \text{fun}(a)$
prima edicola fun
e poi voluto a

ESPRESSIONI IN IMP

Exp $E \rightarrow A \mid B$ ← aritmetico
← booleano
 $A \rightarrow \underline{N} \mid \underline{A \text{ op } A}$ ← numero $\mathbb{Z}$
← $op \in \{*, -, +\}$
 $B \rightarrow \text{true} \mid \text{false} \mid \text{not } B \mid B \text{ or } B \mid A = A$

Semantica (dinamica) di Exp $\rightarrow$ valutazione

$\mathcal{E}$ insieme delle espressioni

$\mathcal{N}$ insieme dei numerali $m, n, p \in \mathbb{Z}$

$\Gamma = \mathcal{E} \quad T = \mathcal{N}$

trasparenza
 $\langle \Gamma, \rightarrow, T \rangle$
 config. config. terminali

$\rightarrow$ definita per induzione strutturale sulla grammatica

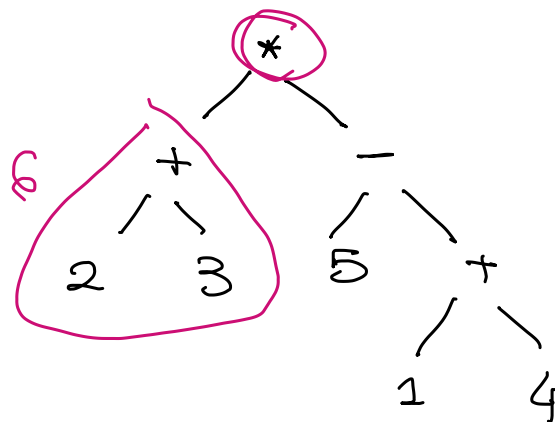
$\Rightarrow_{\varepsilon_1} : \underline{m} \text{ op } \underline{n} \rightarrow \underline{p}$ ← simboli
 $5 + 3 \rightarrow 8$ ← $m \text{ op } n = p$ (numeri)
 $5 + 3 = 8$

$\Rightarrow_{\varepsilon_2} : \frac{e_1 \rightarrow e'_1}{\underline{e_1} \text{ op } \underline{e_2} \rightarrow e'_1 \text{ op } e_2}$

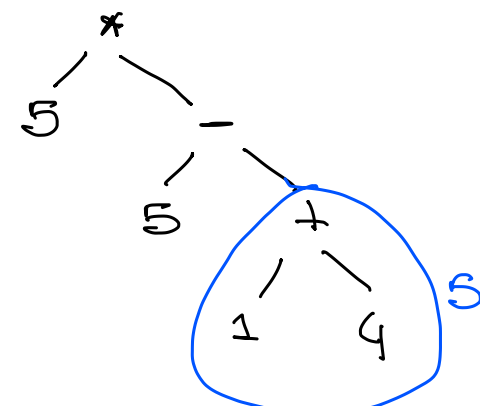
$\Rightarrow_{\varepsilon_3} : \frac{e_2 \rightarrow e'_2}{\underline{m} \text{ op } \underline{e_2} \rightarrow m \text{ op } e'_2}$

$(2+3) \text{ op } (5 - (1+4)) \sim \triangleright$
 $\underbrace{(2+3)}_{e_1} \text{ op } \underbrace{(5 - (1+4))}_{e_2}$

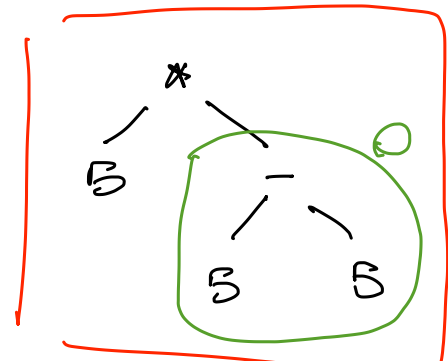
↓
 valore e_1



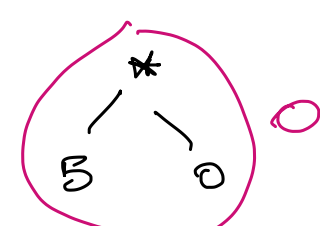
$$\begin{array}{l} \xrightarrow{e_1} (2+3) \rightarrow 5 \\ (2+3) * (5 - (1+4)) \rightarrow \underbrace{5 * e_2}_{m * e_2} \end{array}$$



$$\begin{array}{l} \xrightarrow{e_2} (5 - (1+4)) \rightarrow 5 - 5 \\ m * e_2 \rightarrow \boxed{m * (5 - 5)} \end{array}$$



$$\begin{array}{l} \xrightarrow{e_3} (1+4) \rightarrow 5 \\ 5 - (1+4) \rightarrow 5 - 5 \end{array}$$



$$\begin{array}{l} \xrightarrow{e_4} 5 - 5 \rightarrow 0 \\ 5 * (5 - 5) \rightarrow 5 * 0 \end{array}$$

$$5 * 0 \rightarrow 0$$

applicazioni di regole

$$\begin{aligned} (2+3) * (5 - (1+4)) &\rightarrow 5 * (5 - (1+4)) \\ &\rightarrow 5 * (5 - 5) \\ &\rightarrow 5 * 0 \rightarrow 0 \end{aligned}$$